

Checkpoint 5 | Consumo de API

Documentação – Checkpoint 5: Domain Driven Design Using Java

Autores:

- Murilo Ayabe Severino - RM567479
- Paulo Cavalcante Caroba - RM566667
- Renan da Silva Paulino - RM566610

Curso: Tecnólogo em Análise e Desenvolvimento de Sistemas

Turma: 1TDSPTS – Prof. Fernando Luiz

SÃO PAULO - 2026

Sumário

1. [Descrição da Solução](#)
2. [Objetivo](#)
3. [Endpoints da API](#)
4. [Arquitetura e Stack](#)
5. [Funcionamento da API](#)
6. [Regras de negócio](#)
7. [Requisitos cumpridos no Checkpoint](#)
8. [Como executar](#)
9. [Observações](#)

Total de Páginas: 10

Descrição da Solução

Esta API foi desenvolvida em Java com Quarkus e tem como objetivo calcular, a partir de um endereço de origem (CEP + número) e uma lista de destinos (CEP + número), qual destino possui a menor distância estimada de rota.

Para isso, a aplicação consulta a API ViaCEP para formatar os endereços e, em seguida, consulta a Google Routes API v2, usando o endpoint `computeRouteMatrix`, que retorna as distâncias em metros entre a origem e cada destino. A menor distância é selecionada e retornada ao usuário. Opcionalmente, o resultado pode ser persistido em banco de dados Oracle e posteriormente consultado via endpoint de listagem.

Objetivo

Desenvolver uma API REST em Java capaz de consumir APIs externas, calcular a menor rota entre múltiplos destinos e persistir os resultados em banco de dados Oracle.

Endpoints da API

Checkpoint 5 1.0.0 OAS 3.1

/o/openapi

Checkpoint 5 da materia Domain Driven Design using Java, envolvendo o uso de APIs

Endereco Disponibiliza funcionalidades sobre endereços.

- POST** `/endereco/calculaRotas` Endpoint que calcula a menor rota entre as disponíveis.
- GET** `/endereco/listarRotas` Endpoint que retorna a lista de rotas do banco de dados.
- POST** `/endereco/salvarRota` Endpoint que salva a menor rota no banco de dados.
- GET** `/endereco/{cep}` Endpoint de busca de endereço na Api do Viacep, utilizando o CEP como parâmetro.
- GET** `/endereco/{cep}/enderecoPreparado` Endpoint que mostra endereço formatado pré busca no google maps.

Schemas

- BuscaRotaRequestDTO** > Expand all object
- DestinoDTO** > Expand all object
- ViaCepDTO** > Expand all object

1. Buscar Endereço por CEP

GET `/endereco/{cep}`

Descrição: Busca informações de um endereço na API ViaCEP.

Exemplo:

```
curl -X GET "http://localhost:8080/endereco/01310100"
```

Resposta (200):

```
{
  "cep": "01310-100",
  "logradouro": "Avenida Paulista",
  "unidade": "",
  "bairro": "Bela Vista",
  "localidade": "São Paulo",
  "uf": "SP",
  "estado": "São Paulo",
  "regiao": "Sudeste",
  "ddd": "11",
  "erro": null
}
```

Name	Description
cep * required string (path)	03978180

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/endereco/03978180' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/endereco/03978180
```

Server response

Code	Details
200	Response body <pre>{ "cep": "03978-180", "logradouro": "Rua Batista Fergusio", "unidade": "", "bairro": "Vila Cardoso Franco", "localidade": "São Paulo", "uf": "SP", "estado": "São Paulo", "regiao": "Sudeste", "ddd": "11", "erro": null }</pre> Download

Response headers

```
content-length: 199
content-type: application/json;charset=UTF-8
```

2. Preparar Endereço para Consulta

GET /endereco/{cep}/enderecoPreparado

Descrição: Retorna o endereço formatado para ser usado na busca de rotas.

Exemplo:

```
curl -X GET "http://localhost:8080/endereco/01310100/enderecoPreparado?numero=1000"
```

Resposta (200):

Avenida Paulista, 1000, São Paulo, SP

Name	Description
cep * required string (path)	03978020
numero string (query)	231

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/endereco/03978020/enderecoPreparado?numero=231' \
-H 'accept: text/plain'
```

Request URL

```
http://localhost:8080/endereco/03978020/enderecoPreparado?numero=231
```

Server response

Code	Details
200	Response body Rua João Távora, 231, São Paulo, SP Response headers content-length: 38 content-type: text/plain;charset=UTF-8

3. Calcular Menor Rota

POST /endereco/calculaRotas

Descrição: Calcula a menor rota entre um endereço de origem e múltiplos destinos.

Body de Entrada:

```
{
  "cepOrigem": "01310100",
  "numeroOrigem": "1000",
  "destinos": [
    {
      "cep": "01311100",
      "numero": "500"
    },
    {
      "cep": "01310200",
      "numero": "2000"
    }
  ]
}
```

Resposta (200):

```
{
  "idRota": null,
  "cepOrigem": "01310100",
  "enderecoOrigem": "Avenida Paulista, 1000, São Paulo, SP",

```

```

"cepDestino": "01311100",
"enderecoDestino": "Avenida Paulista, 500, São Paulo, SP",
"distancia": "1.5km"
}

```

Possíveis Erros:

- 400 - CEP inválido, lista de destinos vazia ou falha de validação da requisição
- 404 - Nenhuma rota válida encontrada entre os destinos enviados

Responses

Curl

```

curl -X 'POST' \
  'http://localhost:8080/endereco/calculaRotas' \
  -H 'accept: application/json' \
  -H 'Content-type: application/json' \
  -d '{
    "cepOrigem": "03978020",
    "numeroOrigem": "231",
    "destinos": [
      {
        "cep": "03977010",
        "numero": "100"
      },
      {
        "cep": "03979000",
        "numero": "200"
      },
      {
        "cep": "08420180",
        "numero": "1000"
      },
      {
        "cep": "08220000",
        "numero": "300"
      },
      {
        "cep": "04415100",
        "numero": "700"
      }
    ]
  }'

```

Request URL

http://localhost:8080/endereco/calculaRotas

Server response

Code	Details
200	Response body <pre> { "cepOrigem": "03978020", "enderecoOrigem": "Rua João Távora, 231, São Paulo, SP", "cep": "03979000", "endereco": "Avenida Custódio de Sá e Faria, 200, São Paulo, SP", "distancia": "1.087km" } </pre> Response headers <pre> content-length: 188 content-type: application/json;charset=UTF-8 </pre>

4. Salvar Rota no Banco de Dados

POST /endereco/salvarRota

Descrição: Calcula e persiste a menor rota no banco de dados.

Body de Entrada: (mesmo do endpoint anterior)

Resposta (201 - Created):

```

{
  "idRota": 1,
  "cepOrigem": "01310100",
  "enderecoOrigem": "Avenida Paulista, 1000, São Paulo, SP",
  "cepDestino": "01311100",
  "enderecoDestino": "Avenida Paulista, 500, São Paulo, SP",
  "distancia": "1.5km"
}

```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/endereco/salvarRota' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "cepOrigem": "01102000",
    "numeroOrigem": "57",
    "destinos": [
      {
        "cep": "03977010",
        "numero": "100"
      },
      {
        "cep": "03979000",
        "numero": "200"
      },
      {
        "cep": "08420100",
        "numero": "1000"
      },
      {
        "cep": "08220000",
        "numero": "300"
      },
      {
        "cep": "04415100",
        "numero": "700"
      }
    ]
  }'
```

Request URL

http://localhost:8080/endereco/salvarRota

Server response

Code

Details

201
Undocumented

Response body

```
{
  "idRota": 3,
  "cepOrigem": "01102000",
  "enderecoOrigem": "Avenida Tiradentes, 57, São Paulo, SP",
  "cepDestino": "01311000",
  "enderecoDestino": "Avenida Paulista, 1100, São Paulo, SP",
  "distancia": "6.023km"
}
```

Response headers

```
content-length: 198
content-type: application/json; charset=UTF-8
```

5. Listar Todas as Rotas Salvas

GET /endereco/listarRotas

Descrição: Retorna todas as rotas calculadas e persistidas.

Resposta (200):

```
[
  {
    "idRota": 1,
    "cepOrigem": "01310100",
    "enderecoOrigem": "Avenida Paulista, 1000, São Paulo, SP",
    "cepDestino": "01311100",
    "enderecoDestino": "Avenida Paulista, 500, São Paulo, SP",
    "distancia": "1.5km"
  }
]
```

Possíveis Respostas:

- **204 No Content** - Nenhuma rota salva

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/endereco/listarRotas' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/endereco/listarRotas

Server response

Code Details

200

Response body

```
{
  "enderecoDestino": "Rua Caro-Sacaibu, 500, São Paulo, SP",
  "distancia": "0.803km"
},
{
  "idRota": 2,
  "cepOrigem": "03978820",
  "enderecoOrigem": "Rua João Távora, 231, São Paulo, SP",
  "cepDestino": "03979000",
  "enderecoDestino": "Avenida Custódio de Sá e Faria, 200, São Paulo, SP",
  "distancia": "1.087km"
},
{
  "idRota": 3,
  "cepOrigem": "01102900",
  "enderecoOrigem": "Avenida Tiradentes, 57, São Paulo, SP",
  "cepDestino": "01311000",
  "enderecoDestino": "Avenida Paulista, 1100, São Paulo, SP",
  "distancia": "6.023km"
},
{
  "idRota": 4,
  "cepOrigem": "01109100",
  "enderecoOrigem": "Ponte Cruzeiro do Sul, 821, São Paulo, SP",
  "cepDestino": "01311000",
  "enderecoDestino": "Avenida Paulista, 1100, São Paulo, SP",
  "distancia": "7.19km"
}
]
```

Response headers

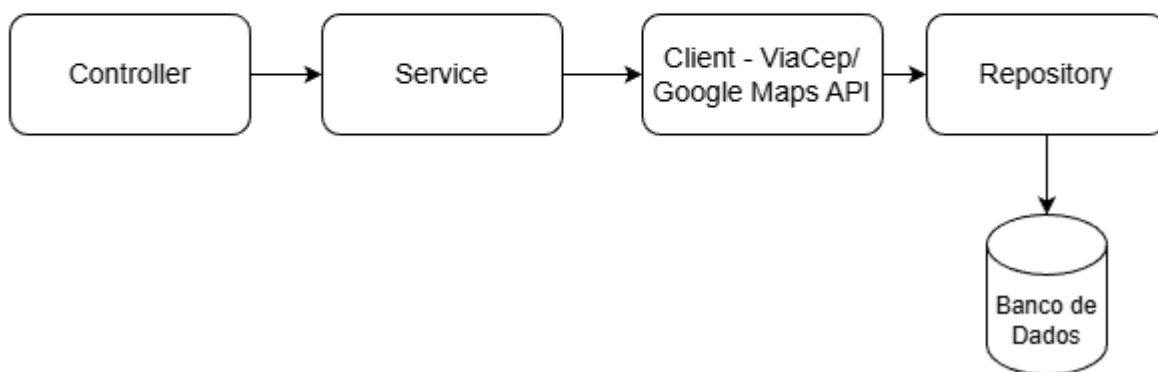
```
content-length: 814
content-type: application/json; charset=UTF-8
```

Arquitetura e Stack

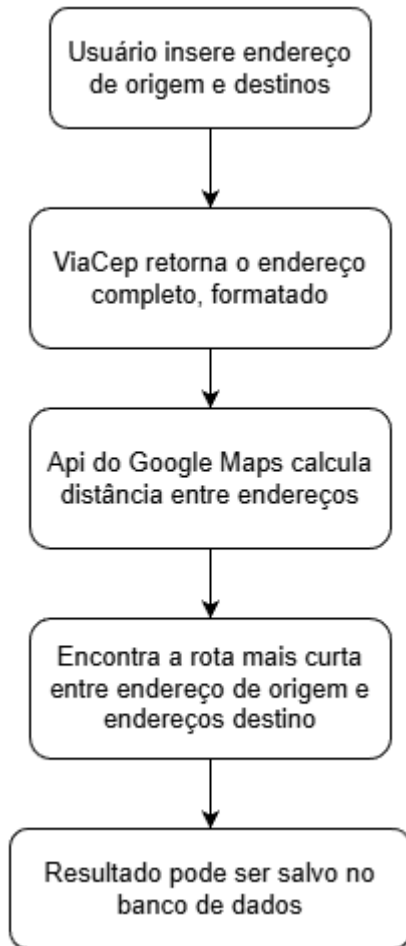
Camadas do Sistema

Arquitetura Simples do Projeto

Controller - camada de interação com o usuário no swagger, chama a Service - que resolve regras de negócio e pode chamar Clients, as Apis externas, podendo assim, depois salvar rota no banco de dados



Fluxo da Aplicação



Tecnologias Utilizadas

- Framework: Quarkus 3.34.6
- Linguagem: Java 21
- Persistência: Hibernate ORM com Panache
- Banco de Dados: Oracle Database
- Documentação: OpenAPI/Swagger
- APIs Externas:
 - Google Routes API v2 ([computeRouteMatrix](#))
 - ViaCEP (Consulta de CEPs brasileiros)

Funcionamento da API

Fluxo de Cálculo de Rota

Validação de CEP:

- Verifica se o CEP possui exatamente 8 dígitos numéricos
- Em caso de CEP inválido, a aplicação retorna mensagem de erro diretamente no endpoint/service

Busca de Endereços

- Consulta ViaCEP para obter logradouro, cidade e UF
- Formata endereço: "Logradouro, Número, Cidade, UF"

Consulta Google Maps

- Envia origem e lista de destinos formatados
- Recebe matriz de distâncias entre origem e destinos
- Filtra apenas rotas com condition: "ROUTE_EXISTS"

Seleção de Menor Rota

- Identifica o destino com menor distância
- Converte metros para quilômetros

Persistência

- Salva ResponseMelhorRotaDTO na tabela rota_completa

Tratamento de Erros

- O projeto realiza tratamentos básicos de exceção para cenários como CEP inválido, ausência de rotas válidas e listas vazias.
- Quando a requisição é inválida, o endpoint responde com 400
- Quando nenhuma rota é encontrada, o endpoint responde com 404
- Quando a lista está vazia, o endpoint de listagem responde com 204

Regras de negócio

- A aplicação considera apenas rotas válidas retornadas pela Google Routes API.
- A menor rota é definida pela menor distância em metros.
- Os endereços são normalizados utilizando a API ViaCEP antes do cálculo.
- O endpoint de cálculo não persiste dados automaticamente.

Requisitos cumpridos no Checkpoint

- Acesso a APIs Externas: Google Routes API v2 + ViaCEP
- Consumo de Dados: Integração com duas APIs externas
- Criação de Endpoints: 5 endpoints REST implementados
- Persistência em Banco: Hibernate ORM com Oracle
- Organização do Projeto: Estrutura em camadas (Controller → Service → Client/Repository)
- Separação de Responsabilidades: Cada camada com função específica
- Tratamento de Dados: Formatação, validação e seleção de menores rotas
- Funcionamento Correto: Integração completa e testável

Como executar

Requisitos

- Java 21+
- Maven 3.8+
- Oracle Database (ou banco compatível)
- API Key do Google Maps

Configuração

Editar application.properties:

ViaCEP

```
quarkus.rest-client.viacep-api.url=https://viacep.com.br/ws
```

Google Maps

```
quarkus.rest-client.google-maps-api.url=https://routes.googleapis.com  
google-api-key=SUA_API_KEY_AQUI
```

Oracle Database

```
quarkus.datasource.db-kind=oracle  
quarkus.datasource.username=system  
quarkus.datasource.password=sua_senha  
quarkus.datasource.jdbc.url=jdbc:oracle:thin:@//localhost:1521/FREEDB1
```

Execução em Desenvolvimento

```
./mvnw quarkus:dev
```

A API estará disponível em <http://localhost:8080>, e o Swagger em <http://localhost:8080/q/swagger-ui>.

Observações

- A API valida CEPs com 8 dígitos numéricos
- Distâncias são retornadas em quilômetros no retorno final
- Mensagens de erro são exibidas de forma simples, direto no endpoint
- Histórico de rotas é persistido no banco para análise